Chương 2

THIẾT KẾ CẤU TRÚC DỮ LIỆU & SƠ ĐỒ HỆ THỐNG

2.1 Mở đầu và Cơ sở lý luận về cấu trúc dữ liệu động trong phần mềm

Trong quá trình nghiên cứu, phân tích và phát triển các hệ thống phần mềm ứng dụng thực tiễn, đặc biệt là các phần mềm quản lý thông tin có tính chất động cao như ứng dụng quản lý danh bạ điện thoại và lịch sử tương tác trên các nền tảng thiết bị, việc lựa chọn cấu trúc dữ liệu đóng vai trò quyết định mang tính sống còn đối với hiệu năng thực thi hệ thống, mức độ tiêu thụ tài nguyên phần cứng (đặc biệt là bộ nhớ RAM) cũng như khả năng mở rộng tính năng trong tương lai.

Danh bạ điện thoại không phải là một tập hợp dữ liệu tĩnh đơn thuần mà là một thực thể thông tin biến đổi liên tục theo thời gian thực. Các thao tác nghiệp vụ cốt lõi của hệ thống bao gồm:

- Thêm mới một liên hệ cá nhân vào danh bạ.
- Tìm kiếm thông tin liên hệ dựa trên từ khóa hoặc tiền tố (prefix search).
- Chỉnh sửa, cập nhật thông tin số điện thoại hoặc tên định danh.
- Sắp xếp toàn bộ danh sách theo thứ tự bảng chữ cái từ A đến Z hoặc ngược lại từ Z đến A.
- Xóa bỏ một liên hệ không còn nhu cầu sử dụng.
- Ghi nhận và truy vấn lịch sử cuộc gọi theo mô hình thời gian thực.

Chính vì tính chất năng động và tần suất thao tác liên tục đó, việc phân tích kỹ lưỡng các ưu nhược điểm của các cấu trúc dữ liệu nền tảng là bước bắt buộc trước khi tiến hành viết mã nguồn.

2.1.1 Hạn chế mang tính bản chất của cấu trúc Mảng (Array)

Trong các giáo trình lập trình cơ bản, Mảng là cấu trúc dữ liệu đầu tiên được tiếp cận. Tuy nhiên, khi áp dụng vào bài toán quản lý danh bạ quy mô lớn, mảng bộc lộ những điểm yếu chí mạng:

1. **Cấp phát tĩnh và sự lãng phí/thiếu hụt bộ nhớ:** Mảng yêu cầu một vùng nhớ liên tục có kích thước cố định tại thời điểm biên dịch hoặc khởi tạo. Nếu lập trình viên dự trù kích thước quá lớn, hệ thống sẽ chiếm dụng và lãng phí một khoảng không gian RAM khổng lồ. Ngược lại, nếu dự trù quá nhỏ, ứng dụng sẽ ngay lập tức sập hoặc gây lỗi tràn bộ nhớ (buffer overflow) khi người dùng nhập số lượng liên hệ vượt quá giới hạn.
2. **Chi phí dời chuyển dữ liệu tuyến tính $O(N)$:** Đặc trưng của mảng là các phần tử nằm liền kề nhau trong bộ nhớ. Khi cần chèn thêm một liên hệ vào giữa danh sách (để duy trì thứ tự bảng chữ cái) hoặc khi xóa một liên hệ, hệ thống bắt buộc phải thực hiện việc dịch chuyển toàn bộ các phần tử phía sau. Với một danh bạ chứa hàng chục nghìn bản ghi, thời gian chờ đợi cho mỗi thao tác chèn/xóa là không thể chấp nhận được trong các ứng dụng hiện đại.

2.1.2 Ưu thế vượt trội của cấu trúc Danh sách liên kết động

Để khắc phục triệt để các nhược điểm của mảng, các cấu trúc danh sách liên kết ra đời. Thay vì lưu trữ trên một vùng nhớ liên tục duy nhất, mỗi phần tử (Node) được cấp phát động độc lập tại bất kỳ ô nhớ nào trên vùng nhớ Heap và được liên kết với nhau thông qua các con trỏ địa chỉ. Điều này mang lại các lợi ích tuyệt đối:

- Không gian bộ nhớ được cấp phát linh hoạt theo số lượng thực tế của liên hệ (thêm bao nhiêu cấp phát bấy nhiêu).
- Thao tác thêm hoặc xóa phần tử chỉ đòi hỏi việc cập nhật lại giá trị con trỏ liên kết mà hoàn toàn không cần dịch chuyển bất kỳ khối dữ liệu nào, đạt độ phức tạp thời gian tối ưu.

2.2 So sánh chuyên sâu đa chiều các Cấu trúc Dữ liệu

Nhằm minh chứng tính khoa học, tính đúng đắn trong việc lựa chọn kiến trúc phần mềm cho đề tài **ĐT12**, Bảng 2.1 thể hiện góc nhìn so sánh đa chiều dựa

trên các tiêu chí kỹ thuật khắt khe.

Bảng 2.1: Bảng so sánh chi tiết các tiêu chí kỹ thuật giữa các Cấu trúc Dữ liệu

Tiêu chí so sánh	**Mảng (Array)**	**Danh sách liên kết đơn**	**Danh sách liên kết đôi (Được chọn)**
Cơ chế cấp phát	Cố định hoặc tái cấp phát tốn kém chi phí ($O(N)$).	Động, cấp phát từng Node linh hoạt trên Heap.	Động, cấp phát từng Node linh hoạt trên Heap.
Hiệu năng Thêm/Xóa	Chậm ($O(N)$ do bắt buộc phải dời dịch chuyển phần tử).	Nhanh ($O(1)$ nếu đã định vị sẵn con trỏ trực tiếp).	Nhanh ($O(1)$ nếu có con trỏ, dễ cập nhật liên kết 2 chiều).
Khả năng duyệt	Duyệt 2 chiều ngẫu nhiên qua chỉ số chỉ mục ($O(1)$).	Chỉ duyệt được 1 chiều duy nhất từ đầu đến cuối.	Duyệt 2 chiều linh hoạt (Từ đầu đến cuối và ngược lại).
Chi phí bộ nhớ phụ	Không tốn con trỏ liên kết phụ trợ (0 byte).	Tốn thêm 1 con trỏ `next` (8 bytes trên nền tảng x64).	Tốn thêm 2 con trỏ `prev` và `next` (16 bytes trên nền tảng x64).
Độ phức tạp cài đặt	Đơn giản, dễ tiếp cận.	Trung bình, dễ quản lý con trỏ đơn.	Phức tạp hơn (đòi hỏi quản lý đồng bộ 2 con trỏ chặt chẽ).

Qua bảng phân tích so sánh trên, mặc dù Danh sách liên kết đôi có tiêu tốn thêm dung lượng bộ nhớ cho con trỏ `prev`, nhưng nó đem lại ưu thế vượt trội không thể thay thế đối với yêu cầu bài toán: hỗ trợ tính năng **duyệt danh bạ hai chiều theo thứ tự từ A-Z và Z-A** cũng như tối ưu hóa thao tác xóa Node khi đã biết trước địa chỉ mà không cần duyệt lại từ đầu.

2.3 Thiết kế chi tiết Cấu trúc Dữ liệu Danh bạ (Doubly Linked List)

2.3.1 Biểu diễn văn bản và Mô tả cấu trúc dữ liệu

Mô-đun danh bạ lưu trữ thông tin cá nhân của người dùng một cách có tổ chức. Mỗi phần tử trong danh bạ là một thực thể độc lập gọi là một Node, bao gồm hai phần chính:

1. **Vùng dữ liệu (Data Field):** Lưu trữ đối tượng thông tin liên hệ cấu trúc `LienHe`, bao gồm hai thành phần chuỗi ký tự:
   - `hoTen` (`std::string`): Lưu trữ họ và tên đầy đủ của liên hệ.
   - `soDienThoai` (`std::string`): Lưu trữ chuỗi ký tự số điện thoại liên lạc chính xác.
2. **Vùng con trỏ (Pointer Field):** Chứa 2 địa chỉ liên kết bộ nhớ quan trọng:

- `prev` (`Node*`): Lưu địa chỉ của Node đứng liền trước trong danh sách. Nếu là Node đầu tiên (`Head`), giá trị được gán là `nullptr`.
- `next` (`Node*`): Lưu địa chỉ của Node đứng liền sau trong danh sách. Nếu là Node cuối cùng (`Tail`), giá trị được gán là `nullptr`.

Để quản lý toàn bộ chuỗi danh sách liên kết, một cấu trúc bao đóng quản lý `DanhBa` được thiết lập để lưu trữ các mốc ranh giới cốt lõi:

- `head` (`Node*`): Luôn trỏ tới Node đầu tiên mở đầu danh bạ.
- `tail` (`Node*`): Luôn trỏ tới Node cuối cùng kết thúc danh bạ.
- `soLuong` (`int`): Ghi nhận tổng số lượng liên hệ hiện có trong hệ thống.

2.3.2 Mã nguồn C++ định nghĩa Cấu trúc Danh bạ

Listing 2.1: Định nghĩa chi tiết Cấu trúc LienHe, Node và DanhBa bằng C++

```
#include <iostream>
#include <string>

using namespace std;

struct LienHe {
    string hoTen;
    string soDienThoai;

    LienHe() : hoTen(""), soDienThoai("") {}
    LienHe(string ten, string sdt) : hoTen(ten), soDienThoai(sdt) {}
};

struct Node {
    LienHe data;
    Node* prev;
    Node* next;

    Node(string ten, string sdt) {
        data = LienHe(ten, sdt);
        prev = nullptr;
        next = nullptr;
    }
};

struct DanhBa {
    Node* head;
    Node* tail;
    int soLuong;
```

```
    DanhBa() {
        head = nullptr;
        tail = nullptr;
        soLuong = 0;
    }
};
```

2.4 Thiết kế chi tiết Cấu trúc Dữ liệu Lịch sử cuộc gọi (Singly Linked List)

2.4.1 Biểu diễn văn bản và Mô tả cấu trúc dữ liệu

Lịch sử cuộc gọi đóng vai trò ghi nhận các mốc thời gian và thông tin tương tác gần nhất của người dùng. Khác với danh bạ đòi hỏi tìm kiếm phức tạp, lịch sử cuộc gọi chỉ yêu cầu cơ chế ghi nhận liên tục theo mô hình LIFO (Last In First Out - Vào sau ra trước) và truy vấn trực tiếp từ cuộc gọi mới nhất đến cũ nhất. Do đó, việc ứng dụng **Danh sách liên kết đơn** là phương án tối ưu hóa hiệu năng và tiết kiệm tài nguyên phần cứng tuyệt đối.

Mỗi Node trong danh sách lịch sử cuộc gọi (`NodeLichSu`) cấu thành từ:

- `data` (`LichSuItem`): Chứa đựng thông tin chi tiết về cuộc gọi bao gồm:
  - `thoiGian` (`std::string`): Mốc thời gian chính xác thực hiện cuộc gọi.
  - `hoTen` (`std::string`): Tên định danh của đối tượng liên lạc.
  - `soDienThoai` (`std::string`): Số điện thoại liên quan đến cuộc gọi.
- `next` (`NodeLichSu*`): Con trỏ liên kết đơn trỏ tới bản ghi lịch sử cuộc gọi tiếp theo trong quá khứ.

2.4.2 Mã nguồn C++ định nghĩa Cấu trúc Lịch sử cuộc gọi

Listing 2.2: Định nghĩa cấu trúc dữ liệu mô-đun Lịch sử cuộc gọi bằng C++

```
struct LichSuItem {
    string thoiGian;
    string hoTen;
    string soDienThoai;

    LichSuItem() : thoiGian(""), hoTen(""), soDienThoai("") {}
    LichSuItem(string tg, string ten, string sdt)
        : thoiGian(tg), hoTen(ten), soDienThoai(sdt) {}
};
```

```
struct NodeLichSu {
    LichSuItem data;
    NodeLichSu* next;

    NodeLichSu(string tg, string ten, string sdt) {
        data = LichSuItem(tg, ten, sdt);
        next = nullptr;
    }
};

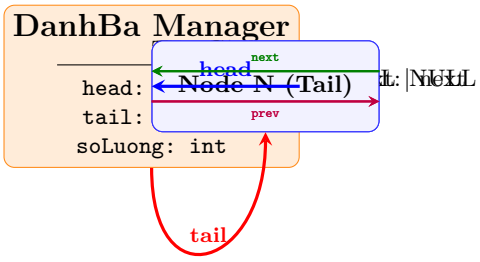
struct LichSuCuocGoi {
    NodeLichSu* head;
    int tongSoCuocGoi;
    int gioiHanToiDa;

    LichSuCuocGoi(int gioiHan = 100) {
        head = nullptr;
        tongSoCuocGoi = 0;
        gioiHanToiDa = gioiHan;
    }
};
```

2.5 Sơ đồ Hệ thống & Cơ chế Liên kết

2.5.1 Sơ đồ tổng thể Cấu trúc Danh sách liên kết đôi (Danh bạ)

Hình 2.1 mô phỏng trực quan cơ chế móc nối con trỏ hai chiều trong mô-đun quản lý danh bạ.



Hình 2.1: Sơ đồ chi tiết cơ chế liên kết hai chiều trong Danh bạ Điện thoại

2.6 Phân tích Thuật toán Thao tác trên Cấu trúc Dữ liệu

2.6.1 Thuật toán Chèn Liên hệ mới vào đuôi Danh bạ

Quá trình thêm mới liên hệ đòi hỏi việc cấp phát động một Node mới và thực hiện điều phối con trỏ theo trình tự logic chặt chẽ nhằm tránh rò rỉ bộ nhớ:

1. Cấp phát động một Node mới P chứa thông tin (HTn, ST) với $P \to prev = nullptr$ và $P \to next = nullptr$.
2. Kiểm tra trạng thái danh bạ: Nếu danh bạ hiện đang rỗng ($head == nullptr$), thiết lập đồng thời $head = P$ và $tail = P$.
3. Nếu danh bạ đã tồn tại các phần tử khác, thực hiện cập nhật liên kết:
   - Móc nối xuôi: $tail \to next = P$.
   - Móc nối ngược: $P \to prev = tail$.
   - Cập nhật dịch chuyển mốc quản lý cuối cùng: $tail = P$.
4. Tăng biến đếm tổng số lượng phần tử: $soLuong = soLuong + 1$.

2.7 Phân tích Vai trò Quản lý của cặp con trỏ Head và Tail

Cặp con trỏ quản lý `head` và `tail` đóng vai trò xương sống quyết định đến độ phức tạp thuật toán của toàn bộ ứng dụng, được tổng hợp tại Bảng 2.2.

Bảng 2.2: Phân tích vai trò kỹ thuật của cặp con trỏ Head và Tail

Con trỏ	**Vai trò kỹ thuật cốt lõi**	**Lợi ích tối ưu hóa hiệu năng**
`head`	- Đánh dấu mốc mở đầu của toàn bộ danh bạ. - Làm điểm neo truy vấn cho thao tác duyệt xuôi và đọc/ghi tệp.	- Cho phép truy xuất phần tử đầu tiên với độ phức tạp $O(1)$. - Giúp thực hiện thao tác xóa/chèn tại đầu đạt $O(1)$.
`tail`	- Đánh dấu mốc kết thúc của danh bạ. - Làm gốc truy vấn cho thao tác duyệt ngược từ Z về A. - Định vị nhanh điểm chèn phần tử mới.	- **Tối ưu hóa chèn cuối:** Giúp thêm liên hệ vào đuôi đạt độ phức tạp $O(1)$ thay vì phải duyệt qua toàn bộ $O(N)$ phần tử nếu không có `tail`.

2.8 Quản lý bộ nhớ và Cơ chế thu hồi tài nguyên (Memory Management)

Một vấn đề cốt lõi khi lập trình với danh sách liên kết động trong C++ là hiện tượng rò rỉ bộ nhớ (memory leak). Mọi vùng nhớ được cấp phát trên Heap thông qua từ khóa `new Node(...)` bắt buộc phải được thu hồi tường minh bằng từ khóa `delete` khi chương trình kết thúc hoặc khi xóa toàn bộ danh bạ.

2.8.1 Thuật toán giải phóng toàn bộ danh bạ

1. Khởi tạo con trỏ tạm `current` trỏ đến `head`.
2. Duyệt qua từng Node: trong khi `current != nullptr`:
   - Lưu trữ Node kế tiếp: `Node* nextNode = current->next;`
   - Hủy vùng nhớ Node hiện tại: `delete current;`
   - Di chuyển sang Node kế tiếp: `current = nextNode;`
3. Đặt lại các mốc quản lý về trạng thái rỗng: `head = nullptr; tail = nullptr; soLuong = 0;`

2.9 Tổng kết Chương 2

Chương này đã trình bày đầy đủ, chi tiết và khoa học cơ sở lý thuyết, sơ đồ kiến trúc hệ thống, bảng so sánh đa chiều, phân tích thuật toán chuyên sâu cũng như mã nguồn C++ chuẩn mực cho các cấu trúc dữ liệu của đồ án. Đây là nền tảng vững chắc để triển khai các thuật toán xử lý nghiệp vụ tại Chương 3.